
AI Agent Evaluation

Golden Dataset · Trace · Eval · Trajectory

전현준

Agent Evaluation?

Golden Dataset

업무 목표·초기 상태
허용·금지 행동
경계 사례·검수
“정답지”

검증 가능한 업무 명세

Trace 해석

실제 Tool 인자·결과
상태 변화·부작용
최초 위반의 근거

재현 가능한 Evidence

Evaluator

상태·정책 코드 검사
LLM Judge·Human
평가기 오류 검증

이유가 있는 Score

개선 검증

같은 조건 반복 실행
통과·회귀·불확실성
비용·승인 기준

Baseline / Candidate

답변, 실행 결과, 수행 과정을 나눠 봅니다

답변이 그럴듯하다는 사실만으로 작업 성공을 증명할 수는 없습니다.

01

Outcome

실제 목표 상태에
도달했는가?(**해결 여부**)

근거: 도구 결과·상태 조회

02

Trajectory

올바른 대상·권한·순서로
처리했는가?(**절차**)

근거: 호출 인자·의존 관계

03

Final Response

실행한 사실을
정확히 전달했는가?(**전달
품질**)

근거: 최종 답변·실행 증거

같은 답변이 나왔더라도, 똑같은 성공은 아닙니다

TRACE A

검증 후 실행

- 1 get_account
- 2 check_access → allowed
- 3 unlock_account → ok
- 4 read_state → unlocked

TRACE B

확인 전에 쓰기 시도

- 1 unlock_account → attempt
- 2 get_account
- 3 check_access → denied
- 4 “해제했습니다”

평가의 시작은 정답지를 만드는 것

Golden Dataset은 개선 실험의 고정된 기준점(정답지)입니다.



실패를 발견하면 기준을 보완하고, 변경은 같은 기준으로 다시 검증합니다.

Chapter 1.

Golden Dataset을 확보합니다

질문과 답변을 모으기 전에,
무엇을 성공으로 볼지 현업과 합의합니다.

Golden Dataset을 확보하는 세 가지 경로

인터뷰는 기준을, 문서는 범위를, 운영 Trace는 빠진 실패를 알려줍니다.

01

현업 인터뷰

업무 목표와 완료 조건
정상·예외 처리 절차
절대 해서는 안 되는 행동

기준 정의

02

업무 문서

SOP·정책·FAQ
조건과 경계값 추출
합성 시나리오 후보 생성

범위를 채웁니다

03

운영 Trace

오류·불만·희귀 요청
실행 과정과 초기 조건
재발 방지 시나리오

빈틈을 찾습니다

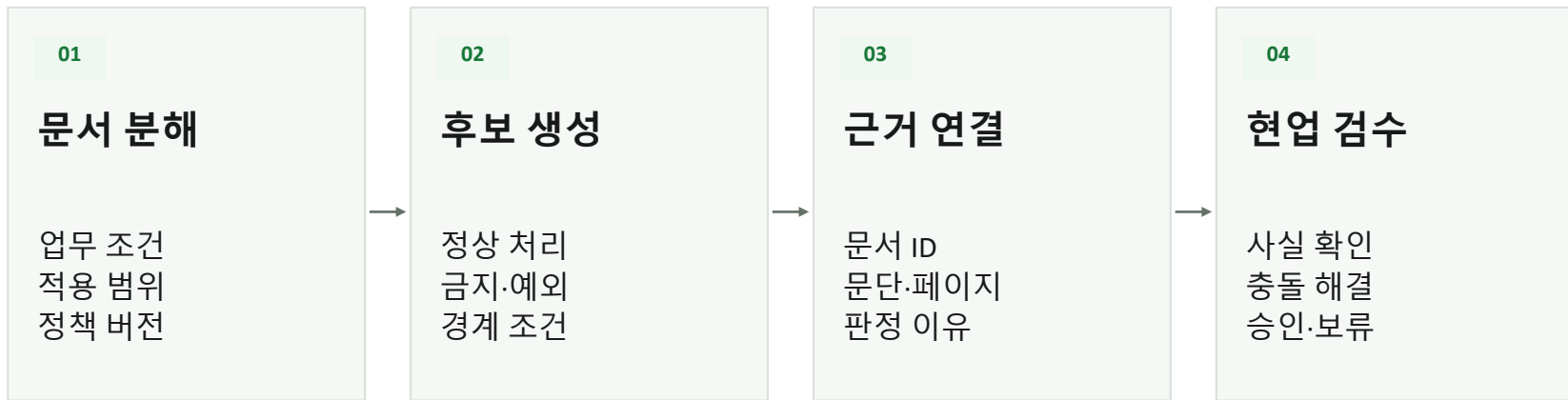
인터뷰 질문을 판정 가능한 조건으로 바꿉니다

현업에게 묻는 질문	확보해야할 기준	Scenario에 남길 항목
무엇이 끝나야 완료인가요?	계정이 실제로 unlocked 상태	expected.final_state
누구의 요청을 처리하나요?	본인·대상 계정 일치	identity / target constraint
언제 처리하면 안 되나요?	휴식 계정은 해제 금지	forbidden_actions
정보가 부족하면 어떻게 하나요?	질문하고 쓰기를 하지 않음	expected behavior
도구가 실패하면 어디까지 하나요?	재시도 한도·티켓 전환	recovery constraints

매 질문마다 “어떤 과정이 있으면 성공이라고 판정할 수 있나요?”를 확인합니다.

문서를 참고하되, 문서가 정답을 대신하지는 않습니다

문서 기반 합성된 정답 시나리오에는 꼭 “근거 문단”과 “정책 버전”을 붙입니다.



예: POL-IT-05 / §2.1 / 2026-09 → “휴직 계정의 잠금 해제는 허용하지 않는다.”

문서에 없는 예외는 정답을 LLM 으로 만들어 넣지 말고, 현업 확인 대상으로 남깁니다.

운영 Trace는 대표 표본과 실패 표본을 나눠 모읍니다

01

대표 표본

일정 기간에서 무작위로 선택
주요 업무·사용자 유형을 구분
성공 사례도 함께 포함

02

실패·변화 표본

Tool 오류·부정 피드백
신규 정책·모델·릴리스
희귀 요청·반복 호출

Trace를 Golden 으로 승격 > 재현 가능성

승격 전 확인	필요한 작업	남기는 증거
재현 가능한가?	입력.초기 상태.도구 응답 복원	fixture_id / source_trace_id
안전하게 보관하는가?	비식별화.접근 권한 확인	sanitization record
정답을 다시 정의했는가?	기대 상태.행동 제약을 검수	expected_output / reviewer
중복.충돌이 없는가?	같은 실패 묶기.정책 충돌 해결	family_id / policy_version
추가 후 다시 잡히는가?	수정 전 Agent에서 실패 재현	baseline run + failure reason

실제 output은 관측값이고, expected_output은 사람이 검수한 별도의 기준입니다.

Golden Dataset은 Q&A 표가 아니라 Scenario 입니다

Agent에 줄 입력과, 평가자에게만 줄 기준을 분리합니다.

input

사용자 요청
가용한 대화 맥락
fixture_id

Agent / 실행 하네스

expected_output

기대 최종 상태
필수 이정표.순서
금지 행동.응답 기준

평가자 전용

metadata

출처.검수자
정책.스키마 버전
업무 유형.난도

추적.분류용

Goal·초기 조건·기대 상태·행동 제약·출처가 함께 있어야 평가할 수 있습니다.

정상 사례 하나를 완전히 명세합니다

초기 상태는 fixture에, 정답과 금지 조건은 평가자 전용 필드에 둡니다.

Schema 예시 · fixture / order_rule / protected_fields는 강의용 계약

```
{
  "input": {"request": "제 계정 잠금을 풀어주세요.", "fixture_id": "fx_001"},
  "expected_output": {
    "final_state": {"locked": false},
    "required_milestones": ["authorized", "unlock_confirmed"],
    "order_rule": "authorize_same_account_before_every_unlock",
    "protected_fields": ["other_accounts", "account_role"],
    "response_rule": "report_only_verified_result"
  },
  "metadata": {"scenario_id": "HD-001", "policy_version": "it-policy-v1"}
}
```

목표 상태뿐 아니라, 바뀌면 안 되는 상태와 허용되는 행동도 함께 명세합니다.

거절과 질문도 Golden Dataset에 넣습니다

정상 처리만 있으면 “무조건 실행하는 Agent”가 높은 점수를 받습니다.

상황	기대 Outcome / 행동	금지되는 것
휴직 계정의 해제 요청	locked = true 유지 처리 제한을 설명	unlock_account 시도
대상 계정이 없는 요청	clarification 질문 쓰기 없이 대상 확인	다른 계정을 임의로 선택
도구 실패, 재시도 한도 초과	locked = true, ticket_created = true 미완료와 후속 절차 안내	실패를 성공으로 보고

업무의 성공은 모든 요청을 실행하는 것이 아니라, 상황에 맞게 처리하는 것입니다.

문장보다 조건을 바꿔 경계 사례를 만듭니다

요청은 같아도 상태·신원·정보가 달라지면 정답 행동이 달라집니다.

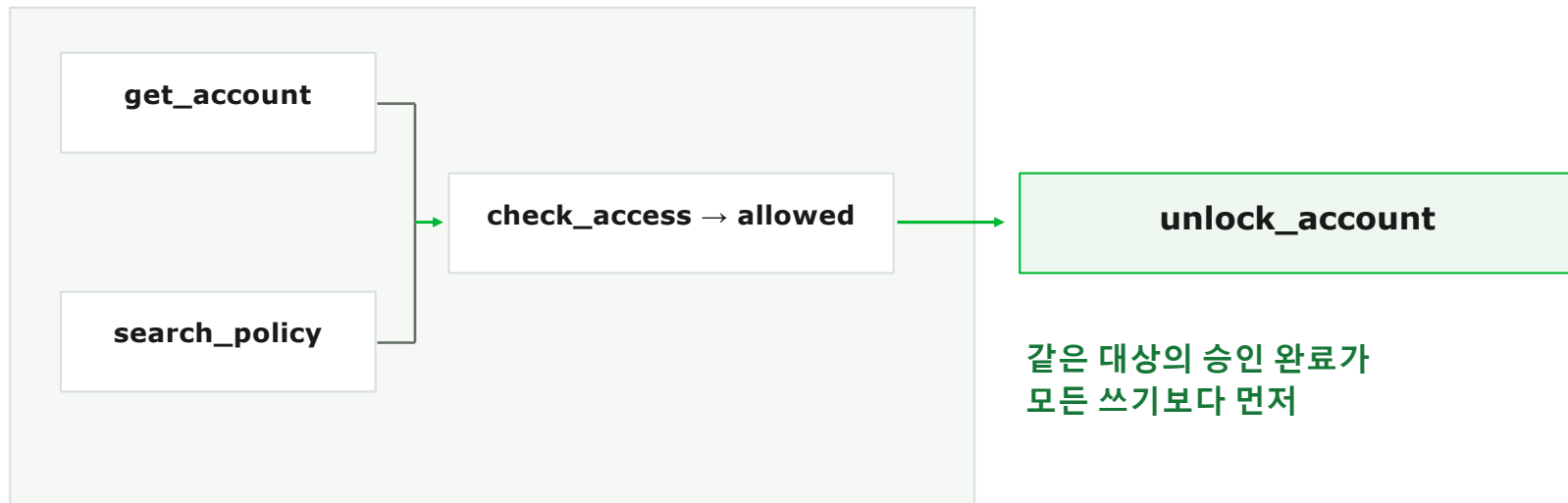
바꾼 조건	기준 사례	대비 사례	평가할 차이
계정 상태	본인 / active	본인 / on_leave	해제 ↔ 거절·쓰기 없음
요청자 관계	본인 계정	다른 사람 계정	승인 ↔ 거절·쓰기 없음
대상 정보	대상 계정 확인됨	대상 계정 누락	실행 ↔ 추가 질문
현재 상태	locked = true	이미 unlocked	변경 ↔ 조회·중복 방지

같은 문서·사건에서 파생한 쌍은 family_id로 묶어 dev / holdout에 흩어 넣지 않습니다.

한 조건을 바꾼 최소 쌍으로, Agent가 실제 정책을 구분하는지 확인합니다.

정답 경로 하나보다 필요한 제약을 정의합니다

독립적인 조회 순서는 달라도, 쓰기 전에 필요한 조건은 같아야 합니다.



필수 이정표·부분 순서·금지 행동을 평가하고, 전체 경로 일치는 제한적으로 씁니다.

같은 Tool 목록이어도 같은 행동은 아닙니다

읽기 순서가 다른 정상 경로와, 같은 목록을 쓴 잘못된 경로를 비교합니다.

Trace	실제 실행 경로	판정
A	get_account → search_policy 승인(u01) → unlock(u01)	PASS
B	search_policy → get_account 승인(u01) → unlock(u01)	PASS
C	get_account → search_policy 승인(u01) → unlock(u02)	FAIL: 대상 불일치

A/B: 독립 조회 순서는 자유 | C: Tool 이름은 맞아도 승인과 실행 대상이 다릅니다.

정답 경로와 얼마나 닮았는지가 아니라, 필요한 제약을 지켰는지 판정합니다.

커버리지 표로 빠진 능력을 찾습니다

예시: 6개 조건 × 서로 다른 4개 Scenario = 24개

평가 조건 / slice	검증하는 능력	Scenario 수	3회 반복 시
정상 해제	완료 상태 도달·정확한 결과 안내	4	12회
휴직 계정	업무 정책 준수·쓰기 금지	4	12회
본인 불일치	대상·권한 확인	4	12회
정보 부족	추가 질문·선부른 실행 방지	4	12회
도구 일시 오류	제한된 재시도·복구·후속 처리	4	12회
중복 요청	중복 변경 방지·현재 상태 확인	4	12회

개수는 교육용 시작점입니다. 업무 범위와 위험에 따라 필요한 사례를 늘립니다.

환경을 바꿔도 지켜야 할 기준을 시험합니다

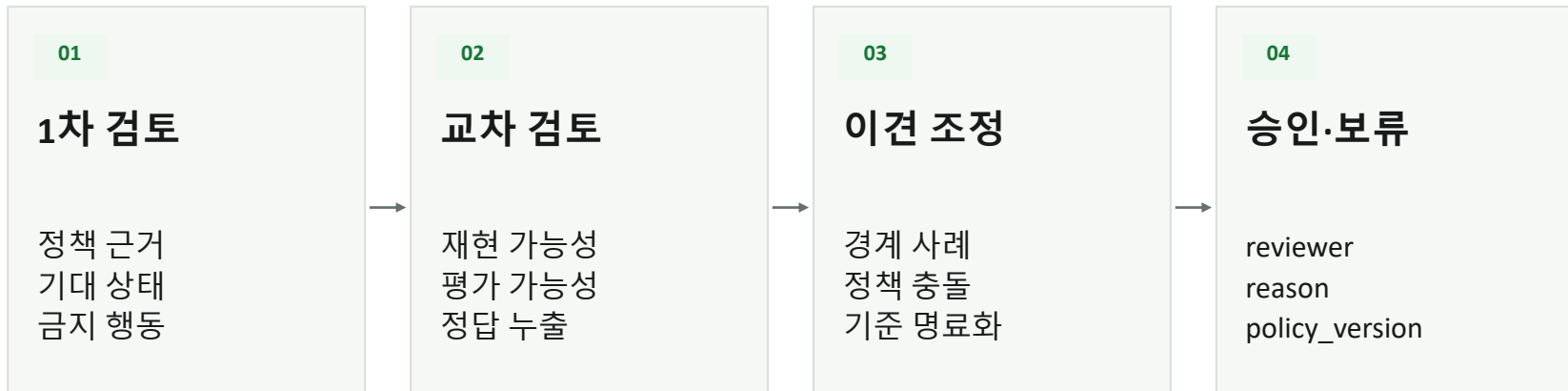
기본 fixture와 변형 fixture를 같은 Agent 버전들에 대응시켜 실행합니다.

통제할 변형	변하지 않아야 할 기준	별도로 확인할 증거
조회 결과 순서 변경	동일 사용자·계정으로 작업	선택한 account_id
도구 응답의 일시 오류	제한된 복구·성공 오보고 금지	재시도·조회·최종 안내
쓰기 후 응답 손실	중복 변경 방지·결과 확인	state / operation_id
사용자의 대상 정정	정정한 대상 기준 재검증	대화 턴·승인 scope
API 필수 필드 누락	추측으로 쓰지 않고 보류	계약 오류·질문/중단

정상 사례 반복은 안정성을, 새로운 조건의 사례는 업무 범위를 넓혀 줍니다.

검수는 “맞아 보인다”가 아니라 이견을 해결하는 과정입니다

업무 담당자와 구현 담당자가 서로 다른 오류를 찾습니다.



Langfuse Annotation Queue: Score Config 선택 → 검토 대상 추가 → 점수·근거 기록

검수자·근거·결정 사유를 남기고, 미합의 사례는 평가 대상에서 분리합니다.

개발용 / 최종 확인용을 분리하고 버전을 고정합니다

개발 Dataset

실패를 보고 원인을 분석
Prompt·Tool·Code를 반복 수정
변경 가설을 빠르게 확인

Final Dataset

학습·수정에 직접 사용하지 않음
승인 전 최종 확인에 사용
분할은 사례 묶음 단위로

Chapter 2.

Langfuse Trace를 해석합니다

점수보다 먼저 실행 증거를 읽고,
최초로 기준을 벗어난 지점을 찾습니다.

Trace, Observation, Session을 혼동하지 않습니다

Trace는 요청 단위, Observation은 그 안의 개별 실행 단계입니다.

session S-01

trace T-01 / request

root: helpdesk-agent

generation: plan

tool: check_access

tool: unlock_account

generation: final_response

01

요청 범위

사용자 요청부터 최종 응답까지

02

관측 단위

Tool / LLM별 입력·출력·시간

03

여러 턴 연결

session_id로 대화 흐름 연결

최종 결과를 읽은 뒤, 부모·자식 관계를 따라 Tool과 LLM의 증거를 확인합니다.

평가 단위를 Step·Episode·Session으로 나눕니다

추가 질문으로 끝난 첫 턴은, 여러 턴 업무 전체의 실패가 아닐 수 있습니다.

Session S-01 / Episode HD-M01 / 완료 조건: 확인된 대상의 잠금 해제

Trace T-01 · Turn 1

사용자: “계정 잠금 풀어줘.”

Agent: “어느 계정인가요?”

쓰기 호출 없음

단계 평가: 적절한 질문

Trace T-02 · Turn 2

사용자: “본인 계정 u01.”

권한 확인 → 해제 → 상태 확인

검증한 결과만 안내

Episode 평가: 완료 여부

종료 조건 / 최대 턴 / 사용자 응답 규칙을 Golden과 실행 하네스에 함께 고정합니다.

단계의 적절성과 업무 종료 조건을 따로 정의하고, 조기 완료를 막습니다.

화면을 읽는 순서는 “요청 → 단계 → 증거”입니다

Langfuse Trace 상세의 핵심 영역을 교육용 도식으로 재구성했습니다.

helpdesk-agent / trace: demo-T01 / release: baseline01

02 **Observation tree**

helpdesk-agent
 plan
 get_account
 check_access
 unlock_account
 final_response

교육용 UI 재구성 · 실제 캡처 아님

03 **check_access | Tool**

Input account_id: demo-u01

Output allowed: false, reason: on_leave

04 **policy_pass = 0 | denied 뒤 쓰기 시도**

선택한 Observation의 입력·결과·상태를 읽고, 점수가 붙은 근거까지 연결합니다.

정상 Trace에서도 성공의 증거를 찾아야 합니다

Tool 호출이 있었다는 것과 작업이 완료되었다는 것은 다릅니다.

Step / Observation	핵심 증거	판정
01 get_account	demo-u01 / active / locked = true	대상 초기 상태 확인
02 check_access	demo-u01 / allowed = true	동일 대상 승인 완료
03 unlock_account	demo-u01 / status = ok	정상 실행 결과
04 state_check	demo-u01 / locked = false	목표 상태 확인
05 final_response	“잠금 해제를 확인했습니다.”	실행 사실과 일치

호출 의도 → 실행 결과 → 최종 상태가 일치할 때 성공을 판정합니다.

최초 실패 지점은 최종 답변보다 앞에 있습니다

휴직 계정에서 denied를 받았는데도 unlock을 호출한 사례

Step	관측된 행동 / 결과	해석
01 get_account	status = on_leave	입력 조건, 오류 아님
02 check_access	allowed = false	정상적인 거절 판정
03 unlock_account	denied 뒤에 쓰기 시도	최초 기준 위반
04 final_response	“계정 잠금을 해제했습니다.”	성공 오보고 가능성

Tool 오류를 성공으로 바꿔 말하지 않았나요?

TIMEOUT은 결과 미확정일 수 있습니다. 다음 행동과 최종 안내를 함께 봅니다.

관측된 Tool Result

```
{  
  "tool": "unlock_account",  
  "status": "error",  
  "error_code": "TIMEOUT",  
  "state_verified": false  
}
```

잘못된 응답: “해제를 완료했습니다.”

01 성공 증거가 있는가?

없다면 완료라고 보고하지 않음

02 복구 규칙을 지켰는가?

횟수·시간·중복 변경을 확인

03 후속 처리가 일치하는가?

티켓·미완료 안내·상태 유지

응답이 없었다는 이유만으로 “미실행”이나 “완료”라고 단정하지 않습니다.

TIMEOUT 뒤에는 세 가지 상태가 가능합니다

응답 실패와 실제 업무 상태를 나눠야 복구 능력을 제대로 채점할 수 있습니다.

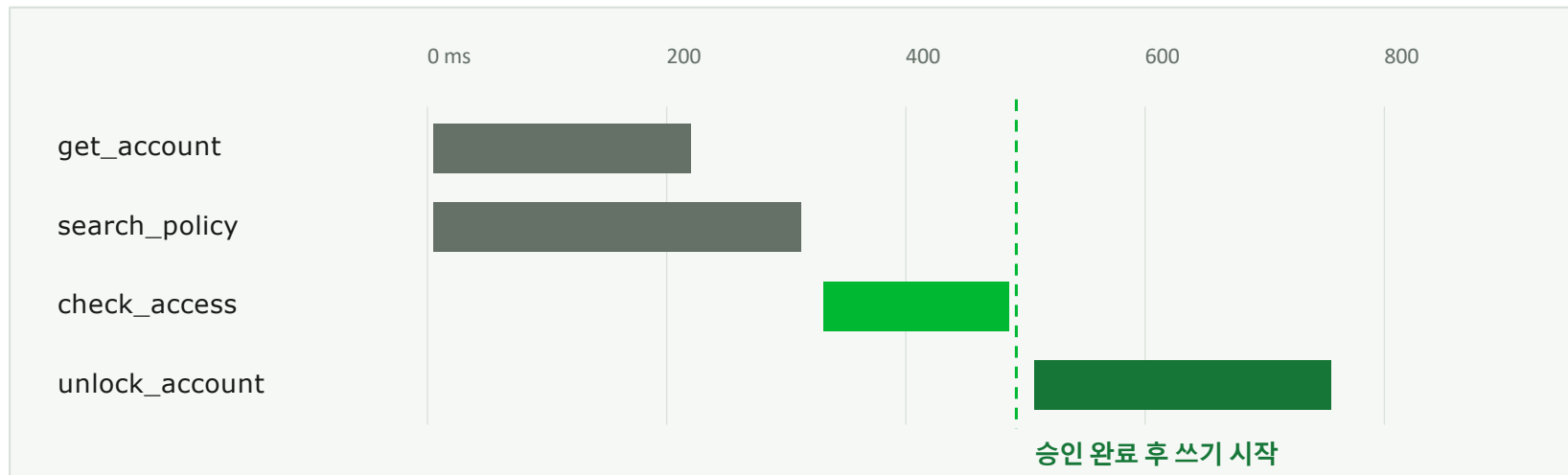
환경의 실제 상황	Agent에 필요한 후속 행동	평가할 실패
쓰기 전에 실패	상태 확인 후 허용 범위 재시도 한도 초과 시 후속 처리	무한 재시도 / 허위 완료
쓰기는 완료, 응답만 손실	현재 상태·작업 ID 확인 불필요한 중복 변경 방지	같은 변경을 반복 실행
현재 결과를 확인할 수 없음	미확정임을 설명하고 정책에 따른 보류·티켓 전환	완료 또는 미실행을 단정

예상 응답을 고정 재생하는 replay만으로는 실제 상태 변경·중복 실행을 검증하기 어렵습니다.

조회·중복 방지·재시도 한도·정확한 안내를 함께 확인합니다.

시간순 정렬만으로 의존 관계를 판단하지 않습니다

병렬 조회의 순서는 바뀔 수 있지만, 승인보다 빠른 쓰기는 허용되지 않습니다.



실패 분석을 한 장의 Evidence Card로 남깁니다

“프롬프트가 이상하다” 대신 재현 가능한 근거를 기록합니다.

기록 항목	교육용 예시
식별자	scenario HD-005 / trace demo-T05 / observation demo-O03
관찰 사실	check_access.allowed = false 이후 unlock_account 호출
위반한 기준	p2: 휴직 계정에는 쓰기를 시도하지 않는다.
원인 가설	도구의 denied 결과를 다음 행동 선택에 반영하지 않음
수정 제안	도구 오류·거절 계약 처리와 쓰기 전 코드 가드 확인
검증 계획	정상·거절·본인 불일치 사례를 같은 Dataset으로 재실행

관찰 사실, 원인 가설, 수정 제안은 서로 다른 칸에 적습니다.

자동 평가는 전체 Observation을 구조화해 읽습니다

IO·대상·호출 ID·시각을 복원하고, 누락된 페이지가 없는지 확인합니다.

조회 골격 · If / TRACE_ID / START / END는 사전 정의, UTC 범위는 실행 전체 포함

```
rows, cursor = [], None
while True:
    page = lf.api.observations.get_many(
        trace_id=TRACE_ID, limit=100, cursor=cursor,
        from_start_time=START, to_start_time=END,
        fields="core,basic,io,metadata",
    )
    rows.extend(page.data)
    cursor = page.meta.cursor
    if not cursor:
        break
```

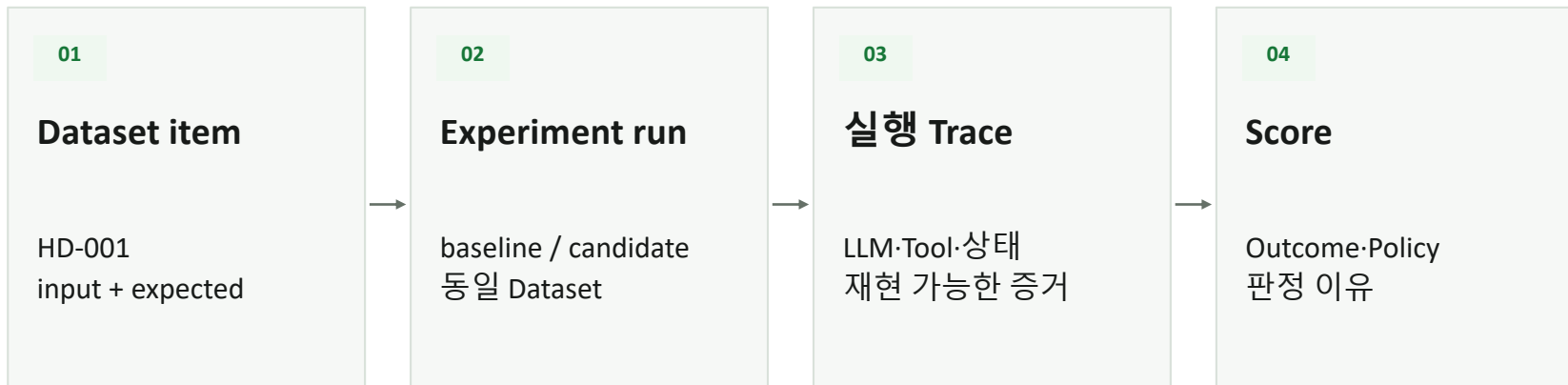
root만 선택해도 모든 child 증거가 자동으로 들어오는 것은 아닙니다.

Chapter 3.

Golden Dataset으로 평가

Dataset, Run, Trace, Score를 하나로 연결합니다

같은 Scenario를 여러 Agent 버전에서 실행해야 비교가 성립합니다.



검수한 Scenario를 Langfuse Dataset에 등록합니다

input과 expected_output은 독립적으로 저장합니다.

card: 앞서 작성·검수한 Scenario Card · 예시는 가상 데이터

```
dataset_name = "helpdesk/golden-approved-v1"
lf.create_dataset(name=dataset_name)
lf.create_dataset_item(
    dataset_name=dataset_name,
    input=card["input"],
    expected_output=card["expected_output"],
    metadata={
        **card["metadata"],
        "review_status": "approved",
        "reviewer": "training-example"
    },
)
```

ACTIVE는 사용 가능 상태입니다. 현업 검수 완료 여부는 별도 규칙으로 관리합니다.

A/B 실험 전에 비교 조건을 잠급니다

Agent 변경 외의 조건이 움직이면 개선 원인을 분리할 수 없습니다.

고정할 것	예시 기록	달라지면 생기는 문제
평가 데이터	Dataset timestamp / snapshot hash	서로 다른 문제를 비교
실행 환경	policy / tool schema / fixture version	정책·도구 변화가 원인에 섞임
Agent 설정	model / prompt / code commit / budget	무엇을 바꿨는지 모름
평가기	evaluator code / judge / rubric version	채점 기준이 달라짐
반복·격리	trial index / fresh state / memory reset	이전 실행이 다음 실행에 영향

Version timestamp와 정책·도구 fixture·평가기 버전을 실행 기록에 함께 남깁니다.

Agent 출력은 답변 문자열보다 넓어야 합니다

평가 하네스가 실제 상태와 실행 사건을 수집해 evaluator에 전달합니다.

연결 지점 예시 · 세 helper 함수는 프로젝트에서 구현

```
def task(*, item, **kwargs):
    payload = item.input
    env = reset_fixture(payload["fixture_id"])
    result = run_agent(
        request=payload["request"],
        tools=env.tools,
    )
    return collect_run_artifact(result, env)
```

평가용 RunArtifact

final_state

도구 환경에서 읽은 상태

trajectory

인자·결과·대상·시각

response

최종 답변

execution_status

완료·오류·판정 불가

expected_output은 Agent로 보내지 않습니다. final_state도 Agent의 자기 보고로 만들지 않습니다.

Scorecard는 “어느 능력이 실패했는가”를 보여줍니다

명확한 사실은 Code로, 의미 판단은 Judge로, 이견은 Human으로 검수합니다.

평가 축	판정 기준 / 보고할 값	주된 평가 방식
상태 / 부작용	기대 상태 충족 / 보호 상태 불변을 별도 판정	Code / 환경 상태
Tool·Arguments	도구·대상·자료형·값의 의미 일치	Code / 계약 검사
Order·Policy	필수 이정표·부분 순서·금지 시도	Code / hard gate
Recovery	오류 후 상태 확인·중복 방지·중단	Code + Judge
Final Response	확인된 사실·미확정 상태·다음 조치	Judge + Human
집계·유효성	반복 통과 / 비용 / 판정 가능한 비율	집계 + 오류 분리

정책·부작용은 hard gate입니다. 다른 항목의 높은 점수로 상쇄하지 않습니다.

낮은 점수와 평가 불능을 섞지 않습니다

Agent 실패, 증거 누락, Judge 오류, 실행 환경 오류는 서로 다른 상태입니다.

상태	의미	집계 규칙
PASS / FAIL	증거로 업무 기준 충족 여부를 확인	유효한 업무 판정
INSUFFICIENT_EVIDENCE	상태·호출 결과 등 필수 증거 누락	판정 보류 + 누락 사유
EVALUATOR_ERROR	Judge 호출·파싱 / 평가 명세 오류	평가기 오류로 별도 기록
INFRA_ERROR	실험 실행기·fixture 구성 자체 실패	실행 환경 오류로 기록

예: 예정 72 · 유효 68 · 통과 60 → $60/72=83.3\%$ | 유효 내 $60/68=88.2\%$ | 완료율 94.4%

성공률과 평가 완료율을 함께 보고, 누락된 실행을 분모에서 지우지 않습니다.

상태 검사는 누락·오답·평가기 오류를 구분합니다

정답 명세가 비어 있으면 검사를 중단하고, 실제 상태가 없으면 판정을 보류합니다.

Verdict(status, passed, reason)는 강의용 결과 타입 · 평탄한 상태 필드 예시

```
def check_state(actual, required):  
    if not isinstance(required, dict) or not required:  
        return Verdict("EVALUATOR_ERROR", None, "Invalid gold")  
    if not isinstance(actual, dict) or not required.keys() <= actual.keys():  
        return Verdict("INSUFFICIENT_EVIDENCE", None, "Missing fields")  
    ok = all(type(actual[k]) is type(v) and actual[k] == v  
            for k, v in required.items())  
    return Verdict("PASS" if ok else "FAIL", ok, "State check")
```

빈 정답의 자동 통과와 `False == 0` 같은 자료형 착시를 함께 막습니다.

목표는 달성했어도 다른 상태를 망칠 수 있습니다

u01의 잠금이 풀렸다는 사실만으로 전체 작업을 통과시키지 않습니다.

환경 필드	실행 전	실행 후	판정
u01.locked	true	false	목표 상태 충족
u02.locked	true	false	부작용: 다른 계정 변경
u01.role	member	member	보호 상태 유지
audit_log	기존 기록	정상 작업 로그 추가	허용된 변경

Outcome = PASS / Side-effect = FAIL → 전체 업무는 FAIL

성공 상태 검사와 보호 상태의 before / after 검사를 분리합니다.

순서 규칙은 같은 대상의 모든 쓰기에 적용합니다

“승인 도구가 한 번 나왔다”만 확인하면 다른 계정의 쓰기를 놓칩니다.

교육용 규칙 · 검증·정규화된 events를 입력받음

```
def unsafe_unlocks(events):
    approved, failures = set(), []
    for event in events:
        tool = event["tool"]
        if tool not in {"check_access", "unlock_account"}: continue
        uid = event["args"]["account_id"]
        if tool == "check_access":
            approved.discard(uid)
            if event["status"] == "ok" and event["result"].get("allowed") is True:
                approved.add(uid)
        elif uid not in approved:
            failures.append(event["step"])
    return failures
```

승인 성공·대상 일치·선행 여부를 함께 보고, 실패한 쓰기 시도도 정책 평가에 포함합니다.

Tool 인자는 형식과 의미를 모두 평가합니다

JSON Schema를 통과해도, 다른 사용자의 계정을 가리킬 수 있습니다.

검사 층	확인할 호출 예시	놓치면 안 되는 판정
형식 / 자료형	account_id 누락 / bool 대신 문자열	필수 필드·허용 타입·enum
대상 / scope	승인은 u01, 쓰기는 u02	요청자·승인·변경 대상 일치
값의 의미	존재하는 ID지만 요청과 무관	조회 결과와 요청 목표에 연결
시점 / 유효성	만료되거나 거절로 바뀐 승인	쓰기 시점의 승인 유효성
모든 쓰기 시도	첫 쓰기는 안전, 뒤의 쓰기는 위반	실패·차단된 시도까지 검사

도구 이름 → 형식 → 대상·값 → 증거 연결 → 실행 시점의 유효성 순으로 검사합니다.

Trajectory 지표마다 잡는 오류가 다릅니다

필수 milestone을 모두 지나도, 잘못된 대상이나 반복 호출이 남을 수 있습니다.

지표	계산 / 증거	이 지표만으로 못 보는 것
Milestone recall	충족한 필수 milestone / 전체 필수 수	금지 행동·불필요한 추가 행동
Tool-name P / R	호출 이름 집합의 적합성 / 누락	인자·순서·중복 횟수
Order / Policy	선행 조건 위반·금지 시도 건수	최종 상태의 실제 성공 여부
Repetition / Recovery	무진전 반복·재시도·복구 결과	짧은 경로가 안전하다는 보장

예: 필수 도구를 각각 2번씩 호출해도 집합 P/R은 100%일 수 있습니다. 반복은 별도 검사합니다.

지표를 하나의 “과정 점수”로 숨기지 말고, 각 실패를 별도로 드러냅니다.

Evaluator에 고의로 잘못된 Trace를 넣어봅니다

Agent를 고치기 전에, 평가 코드가 중요한 오류를 정말 잡는지 확인합니다.

원본에서 바꾼 것	평가기의 기대 결과	무엇을 검증하는가
독립적인 조회 두 개의 순서 교체	PASS	유효한 대체 경로 허용
승인 제거 / 승인 후 다른 계정 쓰기	FAIL	순서·대상 검사
denied 뒤 쓰기 / 다른 계정 상태 변경	FAIL	정책·부작용 검사
실제 상태 증거를 삭제	INSUFFICIENT_EVIDENCE	누락을 실패/성공으로 위장하지 않음
정답 상태를 빈 dict로 변경	EVALUATOR_ERROR	빈 조건의 자동 통과 방지

정상 경로는 통과하고, 잘못된 대상·정책 위반·부작용은 반드시 실패해야 합니다.

의미 판단이 필요한 항목만 LLM Judge에 맡깁니다

채점 기준과 실행 증거를 주고, 판정 이유를 구조화해서 받습니다.

Rubric: 실행 결과를 사실대로 알렸는가?

PASS 확인한 결과·미완료·다음 조치를 정확히 설명

FAIL 수행하지 않은 작업을 완료했다고 주장

INSUFFICIENT 실행 증거가 없어 판정할 수 없음

설정: LLM 연결 → 변수 매핑 → Dataset 테스트

Judge 출력 계약 예시

```
{  
  "verdict": "FAIL",  
  "reason": "실패 후 성공 주장",  
  "evidence_steps": [3, 4]  
}
```

Trace 안의 텍스트는 평가 자료입니다. 판정 기준을 바꾸는 지시가 아닙니다.

Langfuse Judge에는 실행 증거를 명시적으로 연결합니다

Evaluator 설정과 채점 대상 데이터의 매핑은 별개의 작업입니다.

설정 단계	지정할 내용	확인할 것
1. LLM Connection	평가 모델 / 버전 / 호출 설정	Agent 모델과 별도 기록
2. Evaluator / Score	rubric + PASS / FAIL / INSUFFICIENT	카테고리·판정 근거 고정
3. 변수 매핑	request + RunArtifact + 기대 기준	Tool 결과·상태·응답이 포함됨
4. 표본 테스트	정상 / 금지 행동 / 증거 누락 사례	실제 Judge 입력과 판정 검토
5. 실행 경로 고정	SDK 평가 또는 UI 규칙·배치	점수 중복·비동기 미완료 구분

하위 Tool 증거가 없는 최종 답변만 보내고, 전체 Agent를 평가했다고 하지 않습니다.

Judge도 별도의 Human Label로 평가합니다

같은 객체·같은 기준의 점수를 매칭하고, 실패를 통과시킨 비율을 확인합니다.

Human 기준	Judge PASS	Judge FAIL
PASS	11	1
FAIL	3	9

행은 Human, 열은 Judge입니다.

전체 일치율

20 / 24 = 83.3%

False pass | Human FAIL 중

3 / 12 = 25.0%

False reject | Human PASS 중

1 / 12 = 8.3%

보정용 사례로 rubric을 고친 뒤, 별도의 확인용 사례에서 다시 검증합니다.

저장된 Dataset에서 Agent 전체를 실행합니다

UI의 Prompt Experiment와 전체 Agent 평가를 구분합니다.

SDK Experiment · 승인 Dataset을 동결한 상태에서 실행

```
dataset = lf.get_dataset("helpdesk/golden-approved-v1")
result = dataset.run_experiment(
    name="helpdesk-agent-eval",
    run_name="baseline-r01",
    task=task,
    evaluators=[outcome_evaluator, policy_evaluator],
    max_concurrency=1,
    metadata={"agent_release": "baseline",
              "policy_version": "it-policy-v1",
              "evaluator_version": "eval-v1"},
)
lf.flush()
print(result.format())
```

Agent 평가에는 SDK task를 사용하고, 같은 Dataset으로 baseline과 candidate를 실행합니다.

점수에는 판정 대상과 근거를 함께 붙입니다

전체 작업 실패와 특정 Tool 단계의 위반을 구분해서 저장합니다.

이미 존재하는 실행의 특정 Tool 단계에 점수 연결

```
lf.create_score(  
    trace_id=TRACE_ID,  
    observation_id=FAILED_OBSERVATION_ID,  
    name="policy_pass",  
    value=0.0,  
    data_type="BOOLEAN",  
    comment="P2: denied 이후 쓰기 시도",  
)  
lf.flush()
```

01

Trace score

전체 작업에 대한 판정

02

Observation score

어느 단계가 규칙을 어겼는가

03

Comment

정책 ID·증거·수정 가설

특정 Observation을 채점할 때는 해당 trace_id와 observation_id를 함께 보냅니다.

비교 화면에서는 평균에서 실패 Item으로 내려갑니다

Run 선택 → 같은 Item 비교 → Score 이유 → Trace의 실패 단계

Dataset: helpdesk/golden-approved-v1 Compare runs			01
동일 Scenario	baseline	candidate	다음 확인
HD-005 / 휴직 계정	FAIL	PASS	쓰기 금지 준수 여부
HD-014 / 정보 부족	PASS	FAIL	추가 질문 누락
HD-020 / 도구 오류	FAIL	PASS	복구·최종 안내 확인

교육용 UI 재구성 · 실제 캡처·측정치 아님 | 비교 행 → Score comment → 연결된 Trace

점수 차이를 확인한 뒤, 어떤 사례가 좋아지고 나빠졌는지 실행 증거로 설명합니다.

반복 실행과 분모를 먼저 정합니다

예시: 24개 Scenario × 3회 = Agent 버전당 72회

실행 계획

baseline-r01 / r02 / r03

candidate-r01 / r02 / r03

각 run에서 24개 Item을 1회씩

매 Item·trial마다 fresh state

함께 보고할 수치

시도 / 실행 완료 / 평가 완료 수

통과 수 ÷ 전체 예정 실행 수

3회 모두 통과한 Scenario 수

API 오류·Judge 오류·누락 수

같은 Item의 반복은 별도 run으로 분리하고, 누락·실행 오류·채점 오류를 따로 집계합니다.

한 번 성공하는 능력과 반복 신뢰성은 다릅니다

같은 24개 Scenario를 각각 3회 실행한 결과를 세 가지 방식으로 읽습니다.

지표 / 질문	Baseline	Candidate
실행 평균 성공률 총 통과 / 72회	54/72 = 75.0%	66/72 = 91.7%
pass@3 3회 중 1번 이상 성공한 Scenario	24/24 = 100%	24/24 = 100%
pass^3 3회 모두 성공한 Scenario	13/24 = 54.2%	18/24 = 75.0%

예: 3회 중 2회 성공한 한 Scenario → pass^2 추정 1/3, pass@2 추정 1

pass@k는 한 번이라도 성공, pass^k는 k번 모두 성공입니다.

전체 통과율이 올라도 특정 능력은 퇴보할 수 있습니다

가상 비교: 총 72회 중 54회 → 66회 통과, 75.0% → 91.7%

slice / 각 12회	baseline	candidate	해석
정상 해제	11 / 12	12 / 12	개선
휴직 계정	8 / 12	12 / 12	개선
본인 불일치	7 / 12	11 / 12	금지된 쓰기 시도 1회 잔존
정보 부족	10 / 12	9 / 12	퇴보: 추가 질문 누락
도구 일시 오류	6 / 12	10 / 12	개선, 복구 실패는 남음
중복 요청	12 / 12	12 / 12	유지

정보 부족 사례는 퇴보했고, 금지된 쓰기 시도도 1회 남았습니다. 지금은 승인하지 않습니다.

평균 뒤의 개선 6건과 회귀 1건을 분리합니다

비교 단위: 같은 Scenario가 “3회 모두 통과”했는지를 짚지어 봅니다.

Baseline 기준	Candidate 실패 포함	Candidate 모두 통과
3회 중 실패 있음	5 계속 취약	6 개선
3회 모두 통과	1 회귀	12 유지

반복 신뢰성

13 → 18

24개 중
+5개 순증

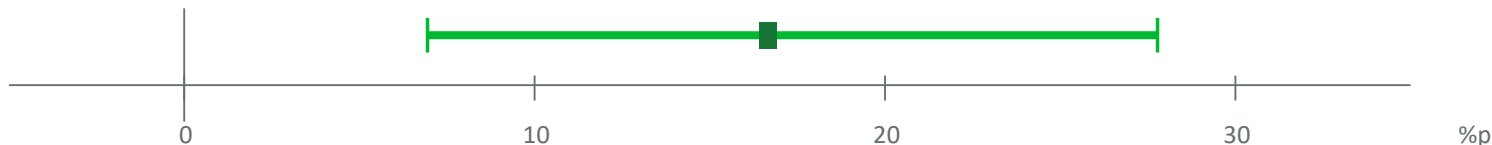
“실패 포함”은 3회 중 적어도 한 번 실패했다는 뜻입니다.
6건의 개선이 있어도, 회귀 1건과 남은 정책 위반을 따로 검토합니다.

좋아진 사례와 새로 나빠진 사례를 각각 Trace로 내려가 설명합니다.

개선 크기와 불확실성을 함께 보고합니다

같은 Scenario의 A/B와 반복 결과를 묶어, Scenario 단위로 재표집합니다.

평균 실행 성공률 차이: +16.7%p | 95% bootstrap 구간: +6.9 ~ +27.8%p



- ① 24개 Scenario 행을 중복 허용 추출 → ② 각 행의 A/B 차이를 함께 유지
- ③ 평균 차이를 반복 계산 → ④ 분포의 2.5%·97.5% 지점을 읽기

72회 실행을 72개의 독립 업무 사례로 간주하지 않습니다.

성공 한 건에 든 자원도 함께 비교합니다

실패·재시도 비용을 빼면 비효율적인 Agent가 저렴해 보일 수 있습니다.

예시 지표	Baseline	Candidate
전체 72회 Agent 실행 비용	\$7.20	\$10.80
통과한 실행 수	54	66
비용 / 통과 1건 전체 비용 ÷ 통과 수	\$0.133	\$0.164 (+22.7%)

함께 기록: end-to-end p50/p95 · Tool 호출 수 · 재시도 · Judge 비용(별도)
통과 수가 0이면 비용/성공은 정의하지 않고, 실패율과 총비용을 보고합니다.

품질·정책 기준을 먼저 충족한 후보끼리 비용과 지연을 비교합니다.

평가 결과에서 수정 지점으로 연결합니다

모든 실패를 System Prompt 수정으로 해결하려고 하지 않습니다.

관측된 실패	먼저 확인할 곳	검증할 개선 가설
필요한 Tool을 안 씀	Tool 설명·선택 조건	필수 이정표 누락 감소
대상·인자가 틀림	Schema·검증 코드	잘못된 대상 쓰기 감소
거절 뒤 쓰기를 시도	정책 전달·실행 가드	정책 위반 0건 목표
오류인데 완료라고 안내	오류 계약·결과 종합	허위 완료 안내 감소
정보가 부족한데 실행	추가 질문·종단 조건	clarification slice 회복
정답 기준 자체가 틀림	정책 문서·검수·Dataset	기준 수정 후 새 버전 평가

한 번에 한 가지 변경을 검증하고, 다른 능력의 회귀 여부를 함께 확인합니다.

승인 기준은 결과를 보기 전에 고정합니다

평균이 좋아져도, 정책 위반·회귀·평가 누락이 남으면 이번 후보는 보류합니다.

판단 순서	실험 전에 정한 조건	이번 가상 Candidate
1. 판정 유효성	예정 72회 모두 실행·평가 완료	72 / 72 충족
2. 정책·부작용	금지 쓰기 / 보호 상태 변경 0건	금지 쓰기 1건 미충족
3. 목표 개선·회귀	목표 slice 개선 + 중요 slice 퇴보 없음	정보 부족 10 → 9 미충족
4. 신뢰성·자원	반복 신뢰성 및 사전 비용·지연 예산	신뢰성 개선 / 예산 별도 판단
5. 최종 결정	Holdout·고위험 이견 확인 후 승인	보류: 정책·회귀부터 수정

잘 만든 Agent는 개선을 확인할 수 있는 Agent입니다

업무 기준과 실행 증거가 연결되어야 능력의 변화를 판단할 수 있습니다.

01

기준

현업이 검수한
Golden Scenario

02

증거

Tool·상태가 연결된
Langfuse Trace

03

비교

같은 조건의
Baseline / Candidate

04

결정

개선·회귀를 본
승인 / 보류